Project:
Predicting Trunk and Branch Masks from Single RGB-D Apple-Tree Images

MMA 3001 REPORT DOCUMENTATION

TAN XUAN YING
34478027

Table of Contents

1. Engineering Problem..... 1
1.1 Context..... 1
1.2 Problem addressed..... 1
1.3 Why this scope..... 1
1.4 Intended use..... 1
1.5 Numerical and machine-learning methods used..... 1
2. Inputs, outputs and data..... 2
2.1 Inputs..... 2
2.2 Outputs..... 2
2.3 The data..... 3
2.4 Data problems found, and how they are handled..... 3
2.5 Invalid, missing and unsupported inputs..... 3
3. Computational approach..... 4
3.1 Pipeline..... 4
3.2 Why a row-wise split..... 5
3.3 Baseline: the mean mask..... 5
3.4 Why this method..... 5
3.5 Alternative: small U-Net..... 6
3.6 Exploratory 3D lift..... 6
3.7 Settings..... 6
3.8 Implementation..... 7
3.9 Assumptions and limitations..... 7
4. Validation method and results..... 7
4.1 What was validated..... 7
4.2 Method and why it is appropriate..... 7
4.3 Metrics..... 8
4.4 Acceptance criteria..... 8
4.5 Results..... 9
4.6 Did validation data influence model selection?..... 10
4.7 What the validation cannot establish..... 10
5. Comparison and optimisation..... 10
5.1 Alternatives compared..... 10
5.2 Optimisation: primal against dual ridge..... 11
5.3 Where the time goes..... 12
5.4 Sensitivity of accuracy to settings..... 12
5.5 Numerical stability and repeatability..... 12
5.6 Trade-offs..... 13
6. Limitations and lessons learned..... 13
6.1 Limitations..... 13
6.2 Lessons learned..... 13
7. AI-use reflection..... 14

7.1 Why AI was used..... 15
8. References..... 16

1. Engineering Problem

1.1 Context

Agricultural robots that prune, thin or harvest apple trees need a compact description of each tree's woody structure (trunk and branches) to decide where to move and what to cut. Colour-and-depth (RGB-D) cameras are cheap, but in summer the foliage hides much of that structure. This is a perception problem for an agricultural robotics application.

1.2 Problem addressed

The extensive request calls for a three-dimensional depiction of apple trees based on RGB-D observations. I focused on a single feasible and verifiable task: anticipate the trunk and branch structure of a tree from one summer RGB-D image, as a 256×256 binary mask, and contrast it with the provided hand-annotated summer ground-truth masks on trees the models haven't encountered. A 3D skeleton is created as an investigative expansion by elevating the pixels of the skeleton using the depth image. It has not been confirmed.

1.3 Why this scope

The supplied data has one view per tree, no camera intrinsics, no depth unit and no 3D ground truth, so multi-view alignment and 3D accuracy cannot be tested. A 2D mask with real ground truth can be.

1.4 Intended use

A mask of this kind would enable a robot to select branch targets and approach routes without needing a complete 3D reconstruction. The accuracy achieved at this point still falls short of that application, and the report discusses the extent to which it can be reliably established (Sections 4 and 6).

1.5 Numerical and machine-learning methods used

Singular value decomposition (SVD) and ridge regression written from scratch, a small convolutional network (U-Net) as an alternative, and profiling and FLOP estimates for performance.

2. Inputs, outputs and data

2.1 Inputs

Each tree has a summer RGB image, a summer depth image and a summer ground-truth mask, identified by the file prefix R<row>N<column> .

Input	Format	Units and domain	Role
Summer RGB image	PNG, 640×480×3, uint8	Intensity 0–255	Appearance
Summer depth image	PNG, 640×480, 8-bit grey	0–255; 0 = no measurement; physical unit unknown	Distance cue and 3D lift
Ground-truth mask	PNG, 256×256, binary	1 = trunk or branch, 0 = background	Training target and validation reference
Tree identifier	File-name prefix	Row and column numbers	Links a tree's files; defines the row-wise split

The model input is a 128×128×4 array (RGB plus depth; depth resized by nearest neighbour so missing-depth zeros are not blended into real values).

2.2 Outputs

Output	Format	Meaning and use
Probability map	256×256, 0–1	Confidence per pixel, thresholded at a value chosen on validation trees only
Predicted mask	256×256 boolean	The trunk/branch estimate

		scored against ground truth
Per-tree metrics	Dice, IoU, boundary F1 (3 px)	Validation evidence
Exploratory 3D skeleton	PLY point cloud, metres (assumed scale)	Indicative shape only; uses an assumed camera, so not validated

2.3 The data

The training dataset contains 562 trees along with all three summer files. The ground-truth masks encompass 5.9% to 10.4% of pixels (median 7.5%). Of the elevated mask pixels on a test tree (R01N01), 99% possess valid depth.

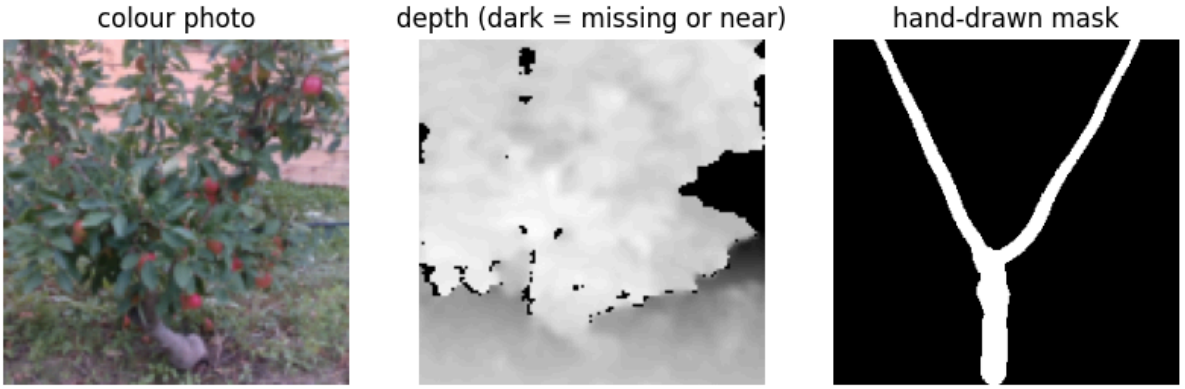


FIgure 1. One tree's photo, depth and hand-drawn mask

2.4 Data problems found, and how they are handled

- 42 winter files carry a different tree ID inside the file name from the one in their prefix. Winter data is therefore not used.
- The brief mentions three seasons; the supplied data has two (summer and winter).
- The mapping of the 256×256 masks onto the 640×480 images is undocumented. Two candidates are tested: stretch (whole image) and centre (central 480×480 crop). The results do not separate them, so the mapping is unconfirmed.
- No camera intrinsics, depth unit or GPS metadata were found.

2.5 Invalid, missing and unsupported inputs

Situation	**Handling**

Dataset folder missing	FileNotFoundError
Folder empty, or duplicate files for a tree	DatasetError
File name off-convention or with a mismatched embedded ID	Reported as an issue; file excluded
Tree missing a summer RGB, depth or mask file	Tree skipped and logged (or error with on_error="raise")
Mask not binary (more than two values) or not 256×256	DatasetError; tree skipped
Depth image all zero	DatasetError; tree skipped
Unreadable image	Tree skipped and logged
Zero-depth pixels in the 3D lift	Dropped
Invalid parameters (alpha ≤ 0, K above the training count, bad split fractions, wrong feature size)	ValueError

The tests exercise these paths. Function-level inputs and outputs are in the docstrings and the generated HTML documentation.

3. Computational approach

3.1 Pipeline

1. Scan and validate the dataset and match files to trees (Section 2).
2. Resize each image to 128×128 and stack RGB and depth into four channels.
3. Split by whole orchard row into training, validation and test sets.
4. Fit a model on the training trees only.
5. Pick the probability threshold with the best mean Dice on the validation trees, then score the test trees once with Dice, IoU and boundary F1.

6. Exploratory only: thin the mask to a skeleton and lift it to 3D.

3.2 Why a row-wise split

Trees in the same row are nearly identical, meaning a random split would likely position almost the same trees in both training and testing, thus artificially boosting the score. Entire rows are allocated to a single set (approximately 20% of rows for testing and 15% for validation; a split of 0 results in 382 / 74 / 106 trees, using rows 3, 5, 7, and 20 as test rows).

3.3 Baseline: the mean mask

Predict the mean of the training masks, thresholded. It uses no image, so any image-based method must beat it to show the image carries information.

Selected method: shape model plus ridge regression.

- *Model of shape.* Training masks are block-averaged to 64×64 and then flattened. Upon removing the mean μ, a slim SVD produces major shapes V_K. A mask is represented by K = 20 coefficients and reconstructed from them. Forecasting ideal coefficients would yield a Dice of approximately 0.72 (the maximum), indicating that the representation is not the limiting factor.
- *Attributes.* The four channels undergo average pooling to create a 16×16 grid (1,024 features) and are standardized using training statistics.
- *Dual form of Ridge.* Using standardized training features A (n × p), target coefficients C, and regularization α = 10^4:

W = A^{\top}\,(A A^{\top} + \alpha I)^{-1}\,(C - \bar{c}) \;=\; (A^{\top}A + \alpha I)^{-1} A^{\top}(C - \bar{c})

Both versions are identical. The dual form addresses an n × n system (n = 382) rather than a p × p system (p = 1,024), which is the lesser one in this case. I developed SVD and ridge from scratch using NumPy instead of relying on a library, and utilized scikit-learn solely in the tests as an external validation.

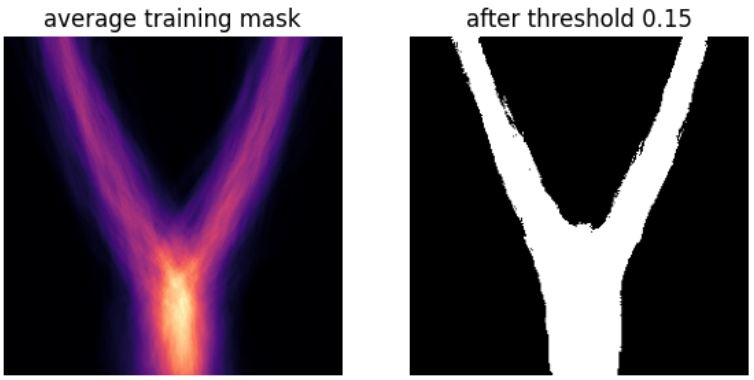


Figure 2. Average training mask and the thresholded version

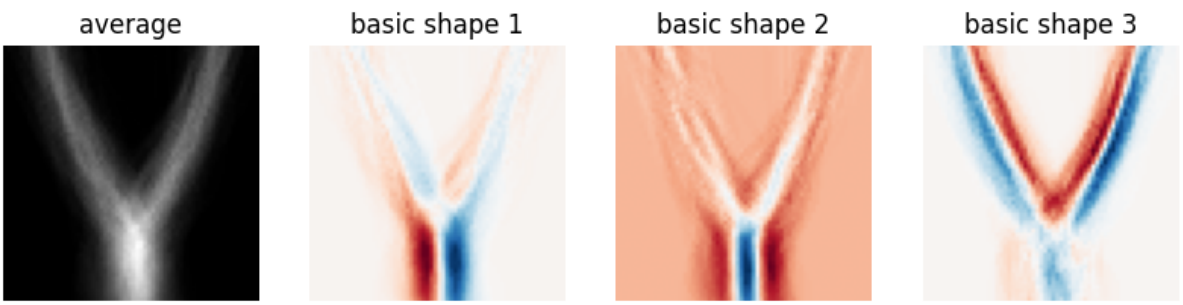


Figure 3. Average mask and the first three basic shaped, plus a small table of K against the ceiling Dice (K-1: 0.259, 2: 0.366, 5: 0.425, 10: 0.544, 20: 0.666, 50: 0.794)

3.4 Why this method

The dataset is limited (around 380 training trees), making it straightforward to validate and inexpensive to replicate across numerous splits using a model with minimal free parameters and a closed-form solution. Its weights are inspectable. Ridge with $\alpha > 0$ remains numerically stable even when the combined features are collinear.

3.5 Alternative: small U-Net

Four down and up levels (channels 16-32-64-128-128), bilinear upsampling, 1,227,665 parameters. Trained using binary cross-entropy alongside soft Dice loss, Adam optimizer, a cosine learning-rate schedule, random horizontal flips, and RGB brightness adjustments (excluding depth). The epoch featuring the highest validation Dice is preserved.

3.6 Exploratory 3D lift

The anticipated mask is reduced to a one-pixel skeleton. A skeleton pixel (u, v) with a depth grey level $d > 0$ transforms into the point below, where s represents the depth scale and the field of view is determined by f_x:

$$z = d\,s,\qquad x = \frac{(u - c_x)\,z}{f_x},\qquad y = -\frac{(v - c_y)\,z}{f_y},\qquad f_x = \frac{W/2}{\tan(\mathrm{HFOV}/2)}$$

The assumed field of view is 60°, and the depth scale is 0.02 m per grey level. The scale adjusts the entire cloud uniformly; the field of view alters its dimensions. A half grey level, approximately 1 cm, limits the quantization error.

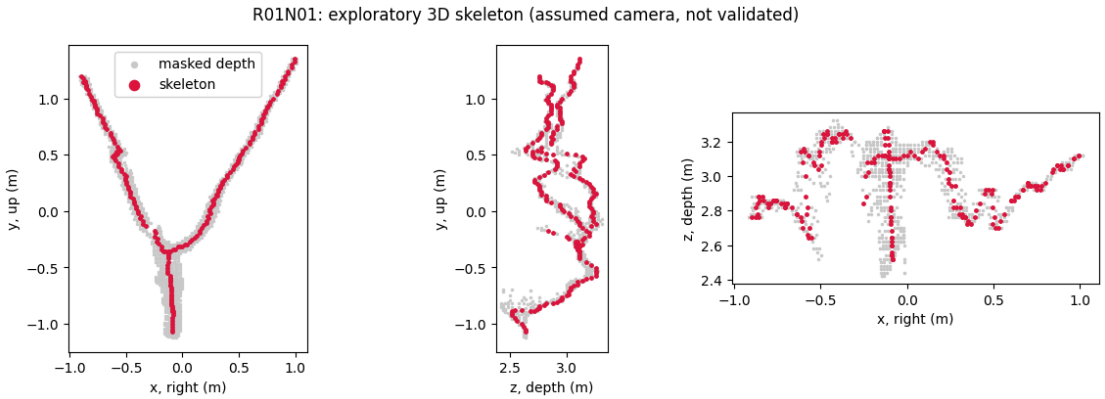


Figure 4. Exploratory 3D skeleton of tree R01N01

3.7 Settings

Setting	Value	Reason
Model input	128×128×4	Keeps the U-Net small and features cheap
Feature grid	16×16	Dice changes by only 0.014 from 4×4 to 32×32
Shape components K	20	Dice saturates at K = 5
Ridge α	10^4	Flat between 10^3 and 10^5
Threshold	Best of 0.1 to 0.9 (step 0.1) on validation	Chosen on validation trees only
Boundary tolerance	3 px	Reported alongside Dice
Split	By row; seed 0, and seeds 0–9 for repeats	Prevents leakage
3D lift (assumed)	HFOV 60°, 0.02 m per level	No intrinsics or depth unit supplied

3.8 Implementation

Python utilizing NumPy, SciPy, Pillow, and scikit-image; PyTorch exclusively for the U-Net. The repository contains a module for each step (data, splits, features, shape model, ridge, metrics, pipeline, U-Net, skeleton, 3D lift, profiling), includes docstrings for all functions, features a script for each experiment that outputs CSV results, and comprises a pytest suite. Every random selection is

initialized with a seed, and images are converted into a cached array once, ensuring that each experiment begins with the same data.

3.9 Assumptions and limitations

The hand-drawn ground-truth masks might miss branches concealed by leaves. The mapping from mask to image has not been verified. Summer depth likely assesses the leaves in front of the limbs. Each tree has a single image, making multi-view fusion unfeasible.

4. Validation method and results

4.1 What was validated

(1) The estimated masks, in comparison to the provided ground-truth masks on trees not utilized for fitting.

(2) The software includes unit tests that validate the numerical methods against external references. The 3D skeleton was not verified: there is no 3D ground truth available.

4.2 Method and why it is appropriate

Element	What it does	Why
Withheld test data	Test trees are never used to fit, to pick the threshold or to pick the epoch	Unbiased estimate on new trees
Row-wise split	Whole rows go to one set	Neighbouring trees are near-copies, so a tree-wise split would leak
Repeated splits	The comparison is repeated over 10 row splits	One test set holds only about four rows, so one split is noisy
No-image baseline	Mean training mask	Shows what the average shape alone achieves
Shape-model ceiling	Dice when coefficients are perfect (about 0.72)	Separates a weak representation from a weak prediction

Sensitivity tests	Alpha, K, feature grid, input channels	Shows whether the result depends on a lucky setting
Unit tests	Ridge against its primal form and scikit-learn Ridge; SVD against scikit-learn PCA; metrics against hand calculations; splits share no row	Checks the code computes what this report says

4.3 Metrics

Averaged Dice and IoU for each tree across the test trees. The masks only cover approximately 7.5% of the pixels, making these scores tough on slender forms. Boundary F1 with a tolerance of 3 pixels is also mentioned, as a branch drawn two pixels away scores nearly zero on Dice but is almost accurate for a robot.

4.4 Acceptance criteria

The selected method must

(a) beat the no-image baseline on Dice in a clear majority of the repeated splits with a gain larger than the split-to-split spread, and

(b) not be worse than the baseline on boundary F1.

4.5 Results

Method	**Test Dice**	**Test IoU**	**Boundary F1**	**Basis**
Mean mask (baseline)	0.368 ± 0.007	0.230	0.270	Mean of 10 row splits; ± is spread over splits
Shape model + ridge	0.391 ± 0.008	0.248	0.273	Same 10 splits
Mean mask, split 0	0.371 ± 0.087	0.231 ± 0.065	not computed	106 test trees; ± is spread over trees
Shape model + ridge, split 0	0.405 ±0.077	Not computed	Not computed	Same 106 test trees

Per-pixel gradient boosting, split 0	0.374 (stretch), 0.378 (centre)	not computed	not computed	One split
Small U-Net, split 0	0.408 ± 0.124	not computed	not computed	One seed; same 106 test trees

- Ridge surpasses the baseline by +0.023 Dice (spread 0.006) and achieves this in all 10 splits. Criterion (a) has been satisfied. The gain from split 0 alone (+0.034) is the highest among the ten, making the average the figure to rely on.
- At boundary F1, the scores are equal (0.273 compared to 0.270). Criterion (b) is satisfied only to the extent that ridge is not inferior: it does not enhance the positioning of branch edges. A prior execution with a coarser threshold grid resulted in ridge having a marginally reduced boundary F1 (0.290 compared to 0.299), indicating that boundary F1 is influenced by the selected threshold, and a 0.003 difference is insignificant.
- On split 0, the ridge (0.405) and U-Net (0.408) scores are nearly identical.
- All trained models are near the baseline and significantly under the 0.67 limit (K = 20).

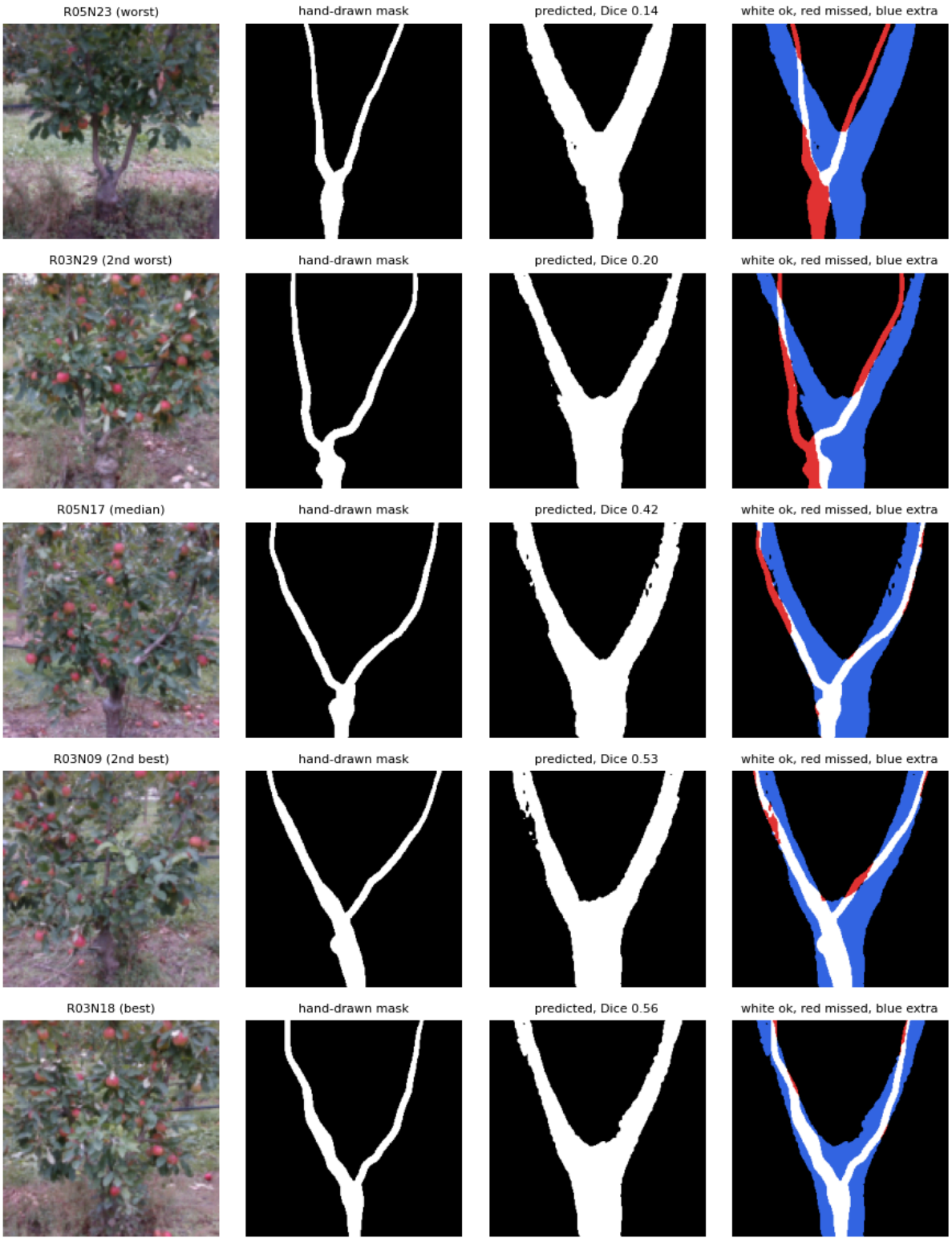


Figure 5. Predictions for the worst, median and best test trees on split 0. White = correct, red = missed by the model, blue = extra.

The model predicts a broad, near-average Y that covers the true branches but is much wider than the

4.6 Did validation data influence model selection?

Yes, in three regulated settings: the chance threshold (optimal from 0.1 to 0.9 via validation Dice), the U-Net epoch (highest validation Dice), and the selection among candidate models. Test trees were utilized solely for end results. The sensitivity outcomes in Section 5 are calculated on test splits, indicating the stability of the result and were not utilized to select settings. The ridge strength was searched from 10^1 to 10^6 and chosen on validation Dice only (10^4 won for every input), and K = 20 was fixed beforehand and not tuned.

4.7 What the validation cannot establish

- The accuracy of the 3D skeleton (no 3D ground truth; field of view, depth scale and mask mapping are assumed).
- That the ground truth is complete: hand-drawn masks may omit branches hidden by leaves, so a good prediction can still score poorly.
- Generalisation beyond one orchard, one summer capture, one camera.
- Independence of the 10 splits: they reuse the same 562 trees, so "10 of 10" is not ten independent trials and the real uncertainty is larger than the reported spread.
- Whether ridge or the U-Net is better: the U-Net was run on one split and one seed.
- Whether stretch or centre is the correct mask mapping.

5. Comparison and optimisation

5.1 Alternatives compared

The mean-mask baseline, pixel-wise gradient boosting, and a compact U-Net were evaluated alongside the chosen shape-model-plus-ridge approach on identical splits and metrics (findings in Section 4).

	Shape model + ridge	**U-Net**
Accuracy (Dice)	0.391 over 10 splits, +0.021 over baseline	0.408 on one split, one seed
Parameters	about 1,024 × 20 weights plus 20 shape components	1,227,665
Arithmetic	about 365 MFLOPs (solve and prediction; excludes the SVD)	about 1.6 GFLOPs per image, forward pass (analytic)
Measured cost	2.4 s and 185 MB peak, on CPU	Not measured

Hardware	CPU	GPU strongly preferred
Interpretability	Weights show which regions drive each shape component	Opaque

The U-Net's higher Dice comes from one split and one seed, and its spread across trees (0.124) is larger than the baseline's (0.087). The data do not yet show it is better than ridge.

Why ridge was selected

The decision is based on consistency rather than the top individual score: it surpasses the baseline in each repeated division, is inexpensive to replicate frequently, is clear, and has low sensitivity to adjustments. The U-Net is said to be a potentially superior option, though evidence isn't provided.

5.2 Optimisation: primal against dual ridge

The initial solution addressed the primal normal equations, a p × p system where p = 1,024. The chosen approach utilizes the dual form, an n × n system where n = 382. Both provide identical weights.

	Primal	**Dual (selected)**
System size	1,024 × 1,024	382 × 382
Leading FLOPs	About 1,533 M	About 352 M
Measured time, best of 5	62.9 ms	10.9 ms
Largest weight difference		3.9e-10

The dual form prevails simply because n < p. Having significantly more trees than features makes the primal form less expensive. In measurements, the dual solve is 5.8 times quicker and requires 4.4 times fewer FLOPs; the speed advantage exceeds the FLOP ratio, likely due to smaller matrices fitting more effectively in the processor cache.

5.3 Where the time goes

The full ridge pipeline takes 2.4 s and 185 MB peak for n = 382, p = 1,024, K = 20. Takes 2.07 s and peaks at 181 MB. Profiling it with cProfile shows where the time goes:

Part of the pipeline	Time in profiled run (s)	Share
SVD of the training masks (shape model)	0.50	32%
Enlarging predicted masks from 64×64 to 256×256 (zoom)	0.49	31%
Scoring masks with Dice (threshold search and test)	0.42	27%
Everything else	0.14	9%

The ridge solve itself (about 11 ms, Section 5.2) does not appear among the 15 most expensive entries. The expensive parts are the shape model, the mask enlargement and the repeated Dice scoring, not the matrix algebra counted in the FLOP estimate. Timings vary a little between runs, and the profiler adds overhead, so the shares are approximate.

5.4 Sensitivity of accuracy to settings

Setting	Values tested (mean test Dice over 3 splits)
Ridge α	10^2: 0.347 · 10^3: 0.383 · 10^4: 0.390 · 10^5: 0.378 · 10^6: 0.373
Shape components K	2: 0.375 · 5: 0.392 · 10: 0.390 · 20: 0.390 · 40: 0.390 · 80: 0.390
Feature grid	4: 0.377 · 8: 0.384 · 16: 0.390 · 32: 0.391
Input channels	RGB: 0.382 · depth: 0.390 · RGB-D: 0.390

Accuracy remains constant across a broad spectrum of configurations, allowing for cost reductions without compromising precision: K can be reduced from 20 to 5 (a fourth of the regression targets), and depth alone matches RGB-D with only a quarter of the input channels. The difference between

RGB alone and depth is roughly equal to one split's range, indicating that depth transmits the signal, but it doesn't confirm it.

5.5 Numerical stability and repeatability

Ridge with $\alpha > 0$ maintains the system matrix as positive definite, ensuring stability in solutions even with collinear features. A minor constant included with each feature's standard deviation avoids division by zero for features with constant values (like depth cells that are always missing); a test checks for this scenario. Every random selection is seeded, and the 10-split outcomes vary by 0.005 to 0.009 in mean Dice.

5.6 Trade-offs

The variations in accuracy among methods are minor in comparison to the variations in cost. The chosen configuration (dual ridge, K ranging from 5 to 20, 16×16 grid) is the most cost-effective option that maintains the complete accuracy achieved.

6. Limitations and lessons learned

6.1 Limitations

- **Accuracy:** All trained models achieve scores ranging from 0.37 to 0.41 Dice, nearly matching the no-image baseline (0.37) and significantly lower than the ceiling of 0.72. The outcome is a slight, steady enhancement in overlap, rather than a solution suitable for robot planning.
- **Boundary placement:** Ridge does not enhance boundary F1, meaning it does not localize branches more effectively than the typical shape.
- **Ground truth:** Hand-drawn masks may omit branches hidden by leaves, which caps the scores that can be reached and makes them hard to read.
- **Data:** One orchard, one summer shot, one perspective for each tree. Winter data is not usable due to incompatible IDs. The mapping from mask to image, the camera intrinsics, and the depth unit are all not specified.
- **3D output:** It relies on a presumed camera and an unverified mapping, as summer depth likely assesses vegetation, making it only suggestive.
- **Statistics:** The repeated splits are not independent, and the U-Net has been run once.

6.2 Lessons learned

1. **Inspect the data before choosing the method:** I began working on a multi-view point-cloud alignment (ICP) initiative. Examining the files revealed a single perspective for each tree, leaving nothing to align, prompting me to alter the issue. Selecting the method initially required time.
2. **One split is not enough evidence:** My initial ridge outcome showed a gain of +0.03 on a single split; across 10 splits, it was +0.02, with roughly 0.008 of the visible gain attributed to the threshold grid utilized in the comparison. Conducting the experiment multiple times across various splits and evaluating both models in the same manner addressed the assertion.
3. **A baseline that ignores the input is essential:** The average mask scores 0.37 due to the similar shapes of trees; without this, a Dice score of 0.39 would have appeared as if learning took place.
4. **Two metrics can disagree.** Ridge raised Dice but not boundary F1, so a single number would have hidden the limit.

5. **Tests catch real mistakes.** A regression test for the row split presumed 23 orchard rows while the data contained 20; running it exposed the mismatch before it reached the report.

7. AI-use reflection

Tools. Claude, through a chat and coding-agent interface, in October 2026.

Task	AI contribution	My contribution
Understanding the brief and datasets, scoping the problem	Medium: explained the brief, proposed the 2D mask task	I selected the apple-tree dataset as it integrated image information with numerical techniques, allowing me to understand the purpose of the output (a tree skeleton).
Notebook code (loading, split, baselines, ridge, U-Net, 3D lift)	High: first versions written by the AI	I executed each notebook cell on Colab with the actual dataset and resolved the path and issues encountered.
Repository, tests, scripts, documentation	High: package structure, tests and scripts written by the AI	I set up the repository, submitted the code in multiple stages with notes explaining every update, and included the README and license. I placed my own name on the license.
Report text	Medium: first drafts of sections	I rephrased every part in my own language, condensed the draft to meet the page limit, and eliminated details I couldn't clarify.

Appleskel package	High	Claude wrote most of the `appleskel` package, tests and scripts.
Running experiments on the real data	None	I ran the codes on colab

In general: The majority of initial code and text drafts were created by AI; I handled the experiments, verified results, and composed all final phrasing.

7.1 Why AI was used.

Due to limited experience with the numerical methods, time pressure, and a way to get working code and structure to learn from.

How AI output was checked.

- Ridge was contrasted with the primal solution and with scikit-learn Ridge; SVD was evaluated against scikit-learn PCA; metrics were compared with manually computed instances; row leakage was examined in the split.
- All the reported figures originate from my own executions of the code on the actual data, and I contrasted the results with those from previous notebook outputs.
- Results from single splits were duplicated across 10 splits prior to being trusted.
- The code that was unexecutable in the AI's environment (the U-Net) was executed by me on Colab.

Decisions that remained mine. accepting the narrowed scope; reporting a modest result honestly; choosing ridge over the U-Net; which settings to report.

Errors, limitations and unhelpful suggestions from the AI.

- Initially, a multi-view alignment (ICP) method was suggested before examining the data; since there is only one view per tree, the concept was abandoned.
- It indicated a ridge increase of +0.03 from one split; successive splits provided +0.02 and adjusted it.
- It initially stated that 3D reconstruction was beyond the scope, then needed to amend that to "unvalidated extension."
- It incorrectly stated that teaching-team approval was required for the project; this requirement pertains to student-proposed projects.
- A test it created assumed 23 rows in the orchard, but the data only showed 20; executing the test revealed this discrepancy.
- The U-Net code was developed without being executed in its own environment, requiring testing from my end.

8. References

Monash University (2026). *MMA3001 Project Brief* and *MMA3001 Project Datasets* (Dataset 1: RGB-D Apple-Tree Reconstruction and Orchard Mapping). Unit documents.

Hoerl, A. E. and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. https://doi.org/10.1080/00401706.1970.10488634

Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press. https://jhupbooks.press.jhu.edu/title/matrix-computations

Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI 2015*, LNCS 9351, 234–241. https://doi.org/10.1007/978-3-319-24574-4_28

Zhang, T. Y. and Suen, C. Y. (1984). A fast parallel algorithm for thinning digital patterns. *Communications of the ACM*, 27(3), 236–239. https://doi.org/10.1145/357994.358023

Hartley, R. and Zisserman, A. (2004). *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press. https://doi.org/10.1017/CBO9780511811685

Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

van der Walt, S. et al. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453. https://doi.org/10.7717/peerj.453

Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*. https://arxiv.org/abs/1912.01703

GITHUB REPOSITORY AND COMMIT SUBMITTED:
https://github.com/XuanYing000/apple-tree-mask-prediction/commit/aa005ac.